

Отчёт по HW3 “Протокол DNS. Адресация уровня L7. L7 <-> L3”

ФИО: Александр Васюков

Группа: БПИ-235

Email: avvasiukov@edu.hse.ru

Декларация об использовании ИИ

Я претендую на оценку до 10 баллов, генеративный ИИ не был использован для работы над этим заданием.

Цель работы

Разработать консольную утилиту на Go, которая вручную формирует Ethernet, IPv4, UDP и DNS пакеты с использованием low-level PCAP API и решает три практические задачи:

- захват и интерпретацию DNS трафика,
- поиск почтового сервиса домена через DNS MX записи и последующее получение IPv4 адресов,
- сравнение ответов корневого DNS сервера и DNS сервера провайдера.

Используемые технологии

- Язык: Go 1.26
- Низкоуровневый сетевой API: github.com/miekg/pcap
- Анализ сетевого трафика: Wireshark
- ОС тестирования и сборки: macOS

Особенности сетевой модели

Приложение рассчитано на работу через физический L2 интерфейс с MAC и IPv4 адресом (en*, eth*).

- Обычный Wi-Fi, мобильная точка доступа и сеть НИУ ВШЭ рассматриваются как один класс сценариев: есть физический интерфейс, scoped DNS и gateway.
- VPN tunnel интерфейсы (utun*, tun*, ppp*, wg*) исключаются из raw Ethernet отправки.
- Если в системе активен VPN и default route указывает на tunnel, приложение выводит предупреждение и продолжает отправлять raw кадры через выбранный физический интерфейс.
- DNS сервера провайдера определяются не глобально, а для выбранного интерфейса.

Для запуска raw PCAP на macOS может потребоваться доступ к /dev/bpf*.

Структура проекта

```
HW-03/  
├─ config.json  
└─ main.go
```

```
|— report.typ
|— report.pdf
|— code_map.md
|— static/
└─ internal/
    ├── arp/
    │   └─ frame.go
    ├── commands/
    │   ├── command.go
    │   ├── sniff.go
    │   ├── mx.go
    │   └─ compare.go
    ├── dns/
    │   ├── message.go
    │   └─ client.go
    ├── packet/
    │   └─ packet.go
    └─ sysinfo/
        └─ netinfo.go
```

Запуск

```
go run . -iface en0
```

При старте приложение выводит:

- выбранный интерфейс,
- локальный IPv4 и MAC,
- gateway,
- DNS сервера провайдера и root DNS,
- признак активного VPN default route,
- список поддерживаемых команд.

Поддерживаются команды:

- sniff [seconds]
- mx <domain> [dns-server-ip]
- compare [root-ip] [provider-ip]
- help
- exit

Реализация подзаданий

1. Захват всех DNS пакетов

Команда: sniff [seconds]

- Интерфейс открывается в promiscuous режиме через `pcap.OpenLive`.
- На уровне ядра устанавливается BPF фильтр `udp port 53 or tcp port 53`.
- Для каждого кадра выполняется ручной разбор:
 - Ethernet,
 - IPv4,

- UDP/TCP,
- DNS header, questions, answers, authority, additional.
- На консоль выводятся MAC/IP/port, DNS ID, флаги, код ответа и краткая сводка по секциям DNS.

2. Поиск D -> IPmx

Команда: `mx <domain> [dns-server-ip]`

Алгоритм выполняется в два шага:

1. Отправляется raw DNS MX query к DNS серверу провайдера.
2. Для каждого найденного MX host отправляется raw DNS A query.

Вывод делается в компактном формате:

```
domain -> mx-host -> ipv4
```

3. Сравнение root DNS и DNS провайдера

Команда: `compare [root-ip] [provider-ip]`

Для доменов `github.com`, `hse.ru`, `draw.io` выполняются A-запросы:

- к корневому DNS серверу с RD = 0,
- к DNS серверу провайдера с RD = 1.

Для каждого ответа выводятся:

- DNS ID,
- флаги,
- rcode,
- next-hop,
- секции Answer, Authority, Additional.

Что возвращает root DNS и что возвращает DNS провайдера

а) Ответ root DNS

Корневой DNS сервер не должен возвращать итоговые IPv4 адреса для `github.com`, `hse.ru` и `draw.io`, потому что он не является авторитетным для соответствующих доменных зон второго уровня. Вместо этого в ответе обычно приходит:

- пустая секция Answer,
- записи NS в секции Authority для соответствующей верхнеуровневой зоны (`.com`, `.ru`, `.io`),
- glue A/AAAA записи в Additional для части этих NS.

Root DNS возвращает referral, то есть указание к каким DNS серверам нужно обращаться дальше.

б) Ответ DNS сервера провайдера

DNS сервер провайдера обычно работает рекурсивно. Поэтому на тот же запрос пользователь получает:

- либо итоговые A записи в секции Answer,
- либо цепочку CNAME + итоговые A,
- при необходимости дополнительные записи в Additional.

По флагам ответа видно, что сервер поддерживает рекурсию (RA = 1).

Экспериментальные режимы среды

Программа спроектирована для четырёх практических сценариев:

1. **Обычный Wi-Fi.** Выбирается активный физический интерфейс с MAC и IPv4, DNS сервера определяются по scoped resolver.
2. **Мобильная точка доступа.** Логика не меняется: интерфейс по-прежнему физический, но адреса gateway и provider DNS будут другими.
3. **Сеть НИУ ВШЭ / общежития.** Также используется физический интерфейс; возможны отличия в provider DNS, ответах recursive DNS и маршрутной конфигурации.
4. **VPN поверх физической сети.** Если default route уходит в tunnel, приложение сообщает об этом, но raw Ethernet кадры формирует через физический интерфейс. Это нужно, потому что utun/tun не предоставляют обычный Ethernet/MAC уровень для данного задания.

Проверка корректности

Реализованы unit-тесты для:

- сериализации и разбора DNS query,
- DNS name compression,
- разбора MX и A resource records,
- checksum IPv4 и UDP,
- обработки битых пакетов без panic.

Команда проверки:

```
go test ./...
```

Скриншоты и подтверждения

Скриншоты с подтверждениями из консоли есть в директории static.

Видео для задачи 3

Ссылка на RuTube:

<https://rutube.ru/video/private/5cd881da13e18f9efadb1077db7ca2b0/?p=Wsf3IuxOmqc6w4ELP6egiQ>

Вывод

Разработана утилита на Go для низкоуровневой работы с DNS поверх PCAP без использования высокоуровневых DNS библиотек. Приложение умеет:

- перехватывать DNS трафик и разбирать его структуру,
- находить MX хосты домена и их IPv4 адреса,
- сравнивать поведение root DNS и recursive DNS провайдера.

Отдельно учтён режим работы при активном VPN: приложение не пытается отправлять raw Ethernet через tunnel интерфейс, а продолжает работу через физический сетевой интерфейс, что соответствует модели задания.